# APAARSetu - Dev 2 (Backend) Complete Work & System Explanation Document

**Role:** Dev 2 (Backend & Verification Engine Lead)

**Project:** APAARSetu (AICTE Student Data Management & Zero-Disclosure Verification Platform)

**Language:** Simple & Clean Hinglish

**Purpose:** Team Sharing, Technical Handoff, aur Presentation Guide

---

## 1. As Dev 2, Mera Exact Role Kya Tha?

Hamari team mein Dev 2 ka responsibility area tha: **Complete Core Backend Development**.

Isme frontend ko serve karne ke liye APIs, database ke sath interaction, AICTE regulatory rules ka engine, bulk CSV upload processing, student data privacy (DPDP Act compliance), aur zero-disclosure verification engine build karna shaamil tha.

Mera pura kaam 9 phases mein divide tha:

- **Phase 0 & 1:** FastAPI setup, Pydantic settings, exception handling, aur Supabase PostgreSQL ke sath 9 SQLAlchemy database models align karna.
- **Phase 2:** Supabase Asymmetric JWT Auth (JWKS) verify karna aur Multi-Tenant Role Based Access Control (RBAC) lagana.
- **Phase 3:** Colleges (Institutions) aur Courses (Programmes) ka CRUD aur Append-Only Security Audit Logging banana.
- **Phase 4:** Student CRUD, Data Privacy Masking (`****1234`), aur Tenant Isolation (College data privacy) build karna.
- **Phase 5:** High-performance Streaming CSV Parser aur 12 AICTE Regulatory Compliance Rules Engine banana.
- **Phase 6:** Batch Review Lifecycle (Inline Error Correction, Revalidation, aur Locking) develop karna.
- **Phase 7:** HMAC-SHA256 Cryptographic Signing, Sliding-Window Rate Limiting (30 req/min), aur Zero-Disclosure Verification Protocol banana.
- **Phase 8:** 15 Automated Pytest Unit & Integration Tests likhna aur 100% green pass karwana.
- **Phase 9:** Production Dockerfile, Render Cloud Deployment Blueprint (`render.yaml`), aur Hackathon Live Demo Walkthrough banana.

---

## 2. End-to-End Pura Backend Kaam Kaise Kar Raha Hai? (The Real Flow)

As Dev 2, maine pure backend ka workflow 4 interconnected pillars par design kiya hai:

### Flow 1: Authentication & Zero-Trust Security Flow

1. User (AICTE Admin, College Admin, Staff, Verifier) frontend par login karta hai.

2. Frontend har API request ke Header mein **`Authorization: Bearer <jwt_token>`** bhejta hai.
3. Backend ka **`security.py`** Supabase ke live JWKS endpoint se public key fetch karke token ko asymmetrically verify karta hai.
4. **Zero-Trust Rule:** Backend token ke claim par bharosa nahi karta, balki database ki **`profiles`** table mein check karta hai ki user ka verified role kya hai aur uska **`institution_id`** kya hai.
5. Agar College A ka admin College B ka data dekhne ki koshish karega, toh backend ka **`check_institution_access`** guard turant **HTTP 403 Forbidden** fek dega.

## Flow 2: Bulk CSV Upload & 12 Rules Engine Pipeline

1. College Admin apne college ke 1,000 ya 10,000 students ki CSV file upload karta hai (**`POST /api/v1/uploads/csv`**).
2. **`csv_parser.py`** memory bachane ke liye streaming chunked format mein file ko parse karta hai aur SHA-256 file checksum nikal kar **`upload_batches`** table mein save karta hai.
3. Engine har student row par **12 AICTE Regulatory Rules** run karta hai:

• APAAR format check (12 numeric digits).

• Age check (15 to 100 years).

• Duplicate Roll Number aur Duplicate APAAR check.

• Central Mock APAAR Registry se match check (kya APAAR ID sach mein exist karti hai?).

4. Uske baad **Batch-level Quota Rule (B01)** run hota hai: Agar college ke B.Tech mein 60 approved seats hain aur file mein 70 students hain, toh system "Intake Quota Exceeded" ka finding raise karta hai.
5. Har error **`validation_findings`** table mein exact row number, rule code, aur error description ke sath save ho jata hai.

## Flow 3: Batch Review, Inline Correction & Locking

1. College admin **`GET /api/v1/batches/{id}/findings`** call karke dekhta hai ki kin-kin rows mein galti hai.
2. Admin ko poori file dobara upload karne ki zaroorat nahi hoti. Woh **`PATCH /api/v1/batches/records/{id}`** call karke usi student ke wrong field (jaise galat DOB ya Roll No) ko edit kar deta hai.
3. Edit hone par **`POST /api/v1/batches/{id}/revalidate`** call hota hai. Engine sirf modified rows ko dobara check karta hai aur findings clear kar deta hai.
4. Jab saari errors resolve ho jati hain, tab **`POST /api/v1/batches/{id}/submit`** call hota hai. Batch permanently **LOCKED (SUBMITTED)** ho jata hai aur students active database mein enter ho jate hain.

## Flow 4: Zero-Disclosure Verification & DPDP Privacy Flow

1. Verifier (Employer/Company) student ko verify karne ke liye **`/api/v1/verify/request`** par request bhejta hai.
2. **Rate Limiting:** Sliding window algorithm verifier ko 30 requests/minute par restrict karta hai taaki koi bot system scrape na kar sake.
3. **Data Privacy (DPDP Act):** System student ka sensitive data (Name, Aadhaar, DOB, Address) verifier ko nahi dikhata.
4. Backend student ke verified status par ek **HMAC-SHA256 Cryptographic Digital Signature** generate karta hai aur verifier ko cryptographically signed proof deta hai:

**`{"verified": true, "status": "VERIFIED", "signature": "e8d4...", "valid_until": "..."}`**.

5. Database aur APIs mein student ka APAAR ID hamesha masked form (**`****1234`**) mein return hota hai.

---

## 3. Har Code File Ka Kya Function Hai? (File-by-File Breakdown)

Aapke pure backend mein kaun si file kya karti hai, uska function-level explanation:

### Core Configuration & Security (`app/core/`)

1. **`app/core/config.py`**:
- **Function:** Pure backend ke environment variables (`.env`) ko load karta hai.
- **Key Class/Functions:** `Settings` (Pydantic BaseSettings class) jo `DATABASE_URL`, `SUPABASE_URL`, `SIGNING_SECRET`, upload limits (`MAX_UPLOAD_MB=5`), aur allowed CORS origins ko type-safe validate karti hai; `get_settings()` function cached settings provide karta hai.

2. **`app/core/security.py`**:
- **Function:** Supabase Auth JWT tokens ko verify karta hai.
- **Key Functions:** `verify_supabase_jwt(token)` — Supabase JWKS (JSON Web Key Set) se public keys fetch karke asymmetric token validation karta hai; fallback mein HS256 secret verify karta hai.

3. **`app/core/errors.py`**:
- **Function:** Global error handling aur standard error envelopes.
- **Key Classes:** Custom exceptions: `AppError`, `NotFoundError` (404), `ConflictError` (409), `ValidationError` (422), `AuthError` (401), `ForbiddenError` (403).
- **`register_exception_handlers(app)`:** Har error ko standard JSON envelope mein format karta hai: `{"error": {"code": "...", "message": "...", "details": ...}}`.

4. **`app/core/masking.py`**:
- **Function:** Data Privacy aur DPDP Act compliance.
- **Key Functions:**
- `mask_apaar_id(apaar_id)`: 12-digit APAAR ID ke pehle 8 digits ko chhipa kar sirf last 4 digits dikhata hai (`123456789012` -> `****9012`).
- `hash_apaar_id(apaar_id)`: Search aur duplicate detection ke liye deterministic SHA-256 hex digest generate karta hai bina real ID leak kiye.

5. **`app/core/signing.py`**:
- **Function:** Zero-disclosure verification ke liye cryptographic signatures.
- **Key Functions:**
- `sign_verification_payload(payload, secret)`: HMAC-SHA256 use karke payload ka tamper-proof digital signature banata hai.
- `verify_signature(payload, signature, secret)`: Timestamp aur content match karke check karta hai ki credential fake ya tampered toh nahi hai.

6. **`app/core/rate_limit.py`**:
- **Function:** Security rate limiter.
- **Key Functions:** `check_rate_limit(key, limit=30, window_seconds=60)` — In-memory sliding window algorithm jo scraping aur brute-force verification ko rokta hai.

7. **`app/core/roles.py`** & **`app/core/constants.py`**:

- **Function:** Application roles (`aicte_admin`, `institution_admin`, `institution_staff`, `verifier`) aur standardized system error codes ka central definition.

---

## Database Layer (`app/db/`)

8. `app/db/session.py`:
- **Function:** Supabase PostgreSQL ke sath SQLAlchemy engine create karta hai, connection pool configure karta hai, aur `SessionLocal` factory banata hai.

9. `app/db/models.py`:
- **Function:** Database ki 9 tables ka Object Relational Mapping (ORM) structure define karta hai:
- `Profile`: System users, unke role aur college linkage.
- `Institution`: AICTE registered colleges, code, state, status.
- `Programme`: College ke courses/degrees aur approved intake quota.
- `UploadBatch`: CSV upload jobs, file checksum, error counts, status (`VALIDATING`, `NEEDS_REVIEW`, `SUBMITTED`).
- `StudentRecord`: Enrolled student details, masked APAAR, category, enrolment date.
- `ValidationFinding`: CSV validation ke dauran nikle granular errors (Row, Rule, Message).
- `VerificationRequest`: Verifiers dwara kiye gaye verification audit logs.
- `ConsentRecord`: Student consent lifecycle (`PENDING`, `GRANTED`, `REVOKED`).
- `AuditLog`: Append-only immutable security ledger.

---

## API Dependency & Guards (`app/api/deps.py`)

10. `app/api/deps.py`:
- **Function:** Dependency injection layer jo har request se pehle security checks run karti hai.
- **Key Functions:**
- `get_db()`: Request ke liye database session open karta hai aur response ke baad cleanly close karta hai.
- `get_current_user()`: Bearer token decode karke database se verified user details aur role laata hai.
- `require_role(allowed_roles)`: Endpoint par role-based restrictions lagata hai.
- `check_institution_access()`: Tenant isolation ensure karta hai taaki ek college doosre college ka data access na kar sake.

---

## Business Services & 12 Rules Engine (`app/services/`)

11. `app/services/csv_parser.py`:
- **Function:** Uploaded CSV bytes ko stream karta hai, headers normalize karta hai, aur row-by-row clean data extract karta hai.

12. `app/services/validation/row_rules.py`:
- **Function:** 12 AICTE Row-level Regulatory Rules:

- **`R01_APAAR_INVALID_FORMAT`**: Exactly 12 numeric digits verify karta hai.
- **`R02_AGE_OUT_OF_BOUNDS`**: DOB se age calculate karke check karta hai (15-100 saal).
- **`R03_INVALID_PROGRAMME`**: Course code validation.
- **`R04_INVALID_CATEGORY`**: Strictly **`OPEN`**, **`OBC`**, **`SC`**, **`ST`**, **`EWS`**.
- **`R05_INVALID_GENDER`**: Strictly **`Male`**, **`Female`**, **`Other`**.
- **`R06_DUPLICATE_ROLL_BATCH`**: Batch ke andar duplicate roll number detection.
- **`R07_DUPLICATE_APAAR_BATCH`**: Batch ke andar duplicate APAAR detection.
- **`R08_FUTURE_ADMISSION_DATE`**: Future dates check.
- **`R09_INVALID_NAME_FORMAT`**: Minimum 2 alphabetic characters check.
- **`R10_INVALID_MOBILE_EMAIL`**: 10-digit mobile aur RFC email regex check.
- **`R11_APAAR_NOT_IN_REGISTRY`**: Central mock registry verification.
- **`R12_REGISTRY_MISMATCH`**: Central registry ke Name/DOB se submitted data ka match.

13. **`app/services/validation/batch_rules.py`**:
- **Function: `B01_APPROVED_INTAKE_EXCEEDED`** — Pure batch ke students ka count programme ke approved intake + supernumerary quota se compare karta hai. Exceed hone par batch-level error raise karta hai.

14. **`app/services/validation/engine.py`**:
- **Function: `ValidationEngine`** class jo context set karti hai, sabhi row rules aur batch rules ko run karti hai, aur saari findings compile karke return karti hai.

15. **`app/services/batch_service.py`**:
- **Function: `revalidate_batch_service`** — Batch ko review stage par laana, corrected records ko re-test karna, findings update karna, aur submit karne par batch ko permanent lock karna.

16. **`app/services/verification_service.py`**:
- **Function: `verify_student_service`** — Rate limiting check karta hai, student record database mein find karta hai, consent status verify karta hai, aur HMAC-SHA256 signed zero-disclosure credential create karta hai.

17. **`app/services/audit.py`**:
- **Function: `log_action(db, ...)`** — Har critical system event (CSV Upload, Batch Submit, Record Edit) ko **`audit_logs`** table mein append-only mode mein save karta hai.

---

## REST API Endpoints (`app/api/v1/routes/`)

18. **`app/api/v1/routes/auth.py`**:
- **`GET /api/v1/auth/me`**: Logged-in user ki verified identity, role, aur institution ID return karta hai.

19. **`app/api/v1/routes/institutions.py`**:
- **`GET /api/v1/institutions`**: Paginated list of colleges (Tenant isolated).
- **`POST /api/v1/institutions`**: AICTE admin dwara naye college ka registration.
- **`GET /api/v1/institutions/me/stats`**: College dashboard ke liye real-time counts (Total Students, Batches, Pending Reviews).

20. **`app/api/v1/routes/programmes.py`**:
- **`GET /api/v1/programmes`**: College ke approved courses aur unki intake capacity list karna.

- **`POST /api/v1/programmes`**: Naya course aur approved quota define karna.

21. **`app/api/v1/routes/students.py`**:
- **`GET /api/v1/students`**: Student directory with pagination aur automatic APAAR ID masking (**`****1234`**).
- **`GET /api/v1/students/{id}`**: Single student record details (RBAC isolated).

22. **`app/api/v1/routes/uploads.py`**:
- **`POST /api/v1/uploads/csv`**: Main entrypoint for bulk CSV upload. Parsing, rules engine execution, batch creation, aur findings generation trigger karta hai.

23. **`app/api/v1/routes/batches.py`**:
- **`GET /api/v1/batches`**: College ke upload batches ki list aur status.
- **`GET /api/v1/batches/{id}/findings`**: Batch ke andar hui galtiyon (findings) ki detailed row-by-row list.
- **`PATCH /api/v1/batches/records/{id}`**: Batch ke kisi student record ko inline fix/edit karna.
- **`POST /api/v1/batches/{id}/revalidate`**: Fixes ke baad batch ko dubara test karna.
- **`POST /api/v1/batches/{id}/submit`**: Batch ko lock karke final student database mein commit karna.

24. **`app/api/v1/routes/verify.py`**:
- **`POST /api/v1/verify/request`**: Verifiers ke liye zero-disclosure verification API jo cryptographic proof return karti hai.

25. **`app/main.py`**:
- Pure application ko initialize karta hai (**`FastAPI(...)`**), CORS middleware setup karta hai, saare v1 routers ko **`/api/v1`** prefix par mount karta hai, aur **`/health`** endpoint serve karta hai.

---

## Automated Tests & Quality Assurance (`tests/`)

26. **`tests/test_upload_flow.py`**:
- Streaming CSV parser aur complete validation engine ka integration test.

27. **`tests/test_validation_rules.py`**:
- AICTE ke 12 rules (valid/invalid APAAR, underage/overage, invalid category, duplicate roll number, intake quota exceed) ka unit test suite.

28. **`tests/test_verify.py`**:
- Masking boundary cases, SHA-256 deterministic hashing, HMAC-SHA256 signature verification, aur signature tampering detection ke tests.
- **Outcome: 15 / 15 Passed (100% Green in 0.44s)**.

---

## Deployment & Infrastructure

29. **`Dockerfile`**: Python 3.13-slim image par optimized production container.
30. **`render.yaml`**: Cloud platform Render ke liye automated build & deployment configuration.
31. **`DEMO_WALKTHROUGH.md`**: 7-minute live presentation sequence.

---

## 4. Team Ko Yeh Kaam Kaise Explain Karein? (Presentation Summary Sheet)

Jab aap apni team ya judges ko apna kaam explain karein, toh yeh points share karein:

1. **"Maine Backend Architecture Zero se Production Ready banaya hai"**:

FastAPI, SQLAlchemy 2.0 ORM, aur Supabase PostgreSQL ke sath clean modular architecture implement kiya hai jisme 9 tables aur complete relational schema mapped hai.

2. **"12 AICTE Regulatory Rules Engine Implement Kiya"**:

Colleges dwara upload kiye gaye data par 12 regulatory checks (Format, Age, Category, Duplicates, Registry matching, aur Quota overflow) real-time execute hote hain aur granular error findings banti hain.

3. **"Complete Review Lifecycle Diya Hai"**:

Colleges ko puri file dobara upload nahi karni padti; woh frontend se inline patch karke revalidate kar sakte hain aur zero-error hone par batch lock kar sakte hain.

4. **"DPDP Act Compliant Privacy & Zero-Disclosure Verification"**:

Database aur APIs mein student ka APAAR ID automatic masked (`****1234`) rehta hai, aur verifiers ko bina personal data leak kiye HMAC-SHA256 cryptographically signed verifiable proof diya jata hai.

5. **"Production Quality & 100% Green Tests"**:

Saara code 15 automated Pytest unit/integration tests se verified hai, Dockerfile aur Cloud deployment config ready hai, aur `main` branch par zero merge conflicts ke sath cleanly merged hai.